Deadlock Analysis of Microservice Systems with Bounded Buffers

1st Li Yang
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
yl2475350477@swfu.edu.cn

2nd Bi Huang
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
125149146@qq.com

3rd XinZhuo Chai
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
takuzhuo@swfu.edu.cn

4th Guoxi Liu
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
guoxiliu@swfu.edu.cn

5th Xiaojing Wu
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
awu@swfu.edu.cn

6th Fei Dai*
School of Big Data and Intelligent Engineering
Southwest Forestry University
CountryKunning,China
daifei@swfu.edu.cn

2024 IEEE Cyber Science and Technology Congress (CyberSciTech) | 979-8-3315-3209-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/CYBERSCITECH64112.2024.00027

***Abstract*—In modern software architectures, microservice are widely used due to their flexibility and scalability. However, the concurrent nature and complex interactions of microservice may lead to problems such as deadlocks, which affect the reliability of the system. In this thesis, we study the deadlock analysis of microservice with bounded buffers. First, we model microservices composed of microservices that communicating asynchronously via bounded FIFO buffers as Labeled Transition Systems. Next, the defined microservices are coded in the PAT (Process Analysis Toolkit) tool using the CSP# language. Finally, the model checking function of PAT was used to detect and analyze the deadlock of the system. The experimental results show that the method effectively identifies potential deadlock problems in the system and provides strong support for the design and optimization of the microservice system.**

Keywords—microservice, bounded buffers, deadlock analysis, Labeled transition systems, model checking

I. Introduction

Microservice architecture is a new development paradigm based on microservices that can be used to build end-to-end microservice systems[1]. Microservice architecture is characterized by flexibility, scalability, lightweight, and loose coupling, which makes many software systems are migrating from monolithic or SOA architecture to microservice architecture[2]. Microservices (or microservices architecture) is a cloud-native architectural approach that contains numerous small, loosely coupled and individually deployable components or services in a single application. These services typically: have their own technology stack, including a database and data management model; and communicate with each other through a combination of REST APIs, event streams, and message proxies. Microservices system consists of a combination of microservices distributed in different containers that can be executed concurrently, and the microservices interact with each other through messages [3].

A microservice system is composed of multiple independent microservices that are deployed in different containers. Each microservice can run independently and can execute tasks concurrently, thus improving the overall efficiency and responsiveness of the system. Microservices interact with each other through messaging, which ensures that each microservice can communicate and collaborate effectively to achieve complex business logic and functionality. Through this architectural design, the microservice system not only has a high degree of flexibility and scalability, but also can better adapt to the changing business requirements and technical environment.

The execution of microservice systems involves a large number of interactions between microservices. The interaction of microservices are complex due to the large number of asynchronous interactions. As an example, Netflix's online service system has to handle 5 billion service calls every day [4]. Tencent's WeChat system contains more than 3,000 microservices running distributed on more than 20,000 machines[5]. Due to the large number of asynchronous interactions in microservices and buffers having a fixed capacity, this can lead to deadlocks during microservice interactions. However, existing research rarely deals with the detection study of deadlock problem. Deadlocks lead to wastage of resources in the microservices system, causing transactions to wait for resources for a long time, resulting in wastage of system resources such as CPU and memory. Our work is to automatically detect the presence of deadlock problem in microservices during interaction with the support of PAT tool.

To be more specific, our approach focuses on analyzing deadlocks in microservice systems that utilize bounded FIFO buffers. The key contributions of this paper can be outlined as follows:

- We model microservices systems with bounded buffers and message set as Labeled Transition Systems (LTSs).
- We implement the proposed method with the support of Process Analysis Toolkit (PAT)[6], we can automatically verify the absence of deadlocks in the system, significantly enhancing the reliability and robustness of our microservice architecture.
- We used 15 cases to validate our proposed methodology. By testing and analyzing these cases, we were able to comprehensively assess the validity and reliability of the methodology.

The rest of this paper is organized as follows. Section II discusses related work. Section III presents the model, including the microservice model and asynchronous combinatorial modeling and giving a case study explanation. Finally, give a notion of deadlock-free property. In Section IV, we discuss our approach, experimental results, and use of model-check tool to verify property values. In Section V, we conclude this paper and discuss further research.

979-8-3315-3209-3/24/$31.00 ©2024 IEEE
DOI 10.1109/CyberSciTech64112.2024.00027

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

II. Related work

As a result of this widespread use, microservices have become a important research areas in recent years. Deadlock analysis in microservice systems has garnered significant attention due to the intricate nature of interactions and dependencies among microservices. Numerous studies have explored various aspects of microservice architecture, focusing on performance, reliability, and fault tolerance. For example, Bao, Liang et al [21] simulated and predicted the performance of microservices and proposed a heuristic microservice scheduling algorithm.， Ueda et al[27] used open-source web service benchmarks to compare the performance of microservice architectures and monolithic architecture were compared for performance. The results show that a microservices architecture incurs significant overhead. Wang et al [28] proposed an elastic microservice scheduling approach that combines the two to minimize the cost of virtual machines while improving system performance. Literature [29] This paper proposes a new combinatorial scaling methodology COPA. based on collected microservice performance data, real-time workloads, expected response times and runtime microservice instance scenarios, COPA uses a queuing network model to compute combinatorial scaling scenarios designed to minimize default and resource costs.

Existing research work on microservices performance enhancement focuses on system performance reliability from the perspective of non-functional requirements. But our work is focused on the functional requirements perspective of microservices and on the interaction reliability of microservice systems. All these research works mentioned above mainly focus on improving the performance of microservice systems and all these methods cannot be used to analyse the interactions in microservice systems.

The field of interaction modeling work has been a great achievement.For example, in [8], Bultan et al. introduced an innovative interaction modeling framework that leverages Büchi automata to effectively model the interactions among Web services. This framework is particularly notable for its use of FIFO (First-In-First-Out) buffers, which ensures that messages are processed in the order they are received. By employing Büchi automata, the framework can accurately capture the complex, asynchronous communication patterns that occur between Web services, providing a robust method for analyzing and verifying their behavior. In [9], Hennicker et al. proposed modeling asynchronous interactions among communicating peers using I/O-transition systems. In[10], Salaün et al. described communicating peers as Labeled Transition Systems. In[11], Dahmani et al. proposed using Petri nets to model asynchronous interactions among communicating peers.

This paper is concerned with interaction errors that occur during the design phase of microservice systems. Errors in the design phase are often difficult to detect easily after the system has been run, and therefore have a greater degree of invisibility and potential harm.

Inspired by these works, we propose a formal verification methodology for deadlocks on microservices that transforms LTS models into a CSP language to model microservices as well as verify the deadlock-free nature of microservices. This approach ensures that microservices interact correctly and aims to enhance the security and reliability of microservices.

III. Method

Figure 1 shows the overview of our approach, we propose a method to determine whether there are deadlocks during the interaction of microservices. It mainly includes the following three main steps: We model microservices systems through microservices communicating asynchronously via FIFO buffers. $CSP\#$ transforms LTS models into microservice systems. Detecting microservices for deadlocks with the support of the PAT tool.

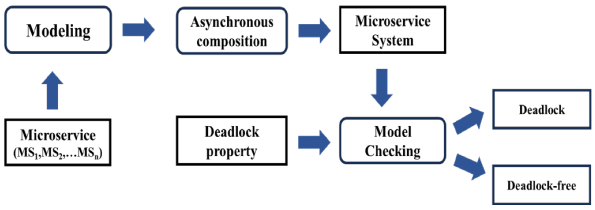


Figure 1: Overview of our approach

In this section, we introduce the modeling language used in this paper and we definition microservices as well as the definition of asynchronous communication via FIFO buffers. In addition, this section examines a case study to explain the modeling of bounded buffers. A brief introduction to microservices communication is provided before we describe the above problem.

In a microservices system, microservices interact with each other using both asynchronous communication [12] and synchronous communication[13]. Synchronous communication occurs when both the sender and receiver coordinate their send-receive actions simultaneously, with the sender remaining blocked until the receiver completes the receive action. In contrast, asynchronous communication allows senders to proceed with subsequent actions immediately after initiating the send, without waiting for the receive action to be completed by the receiver. Furthermore, synchronous interactions can multiply downtime during the operation of microservice systems, which can be detrimental [14]. Therefore, this paper focuses only on asynchronous communication.

In this section, we provide a diagrammatic example of a microservice system with complex interactions based on the example from [15]. Figure 2 illustrates the three communicating microservices MS_1, MS_2, and MS_3 with messages a, b, c, d, e and f. A circle with a half arrow represents the initial state, and a double circle represents being in the terminated state. A circle indicates the current state of the microservice MS, where ! indicates a send and ? indicates a receive. For example, $!e^{2\to3}$ means that microservice MS_2 takes a message a from $buffer_1$ and send it to $buffer_3$ of microservice MS_3. The two numbers subscripted by the $S_{0,1}$ in the circle, the first number indicates the state of the microservice and the second indicates the first microservice. where the subscript 0 indicates the initial state. For example, $S_{0,1}$ to $S_{1,1}$ means from the initial state to another state when microservice MS_1 finishes executing a change. The $buffer_1$ represents the buffers belonging to MS_1, where each individual buffer functions as a First-In, First-Out (FIFO) message queue.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

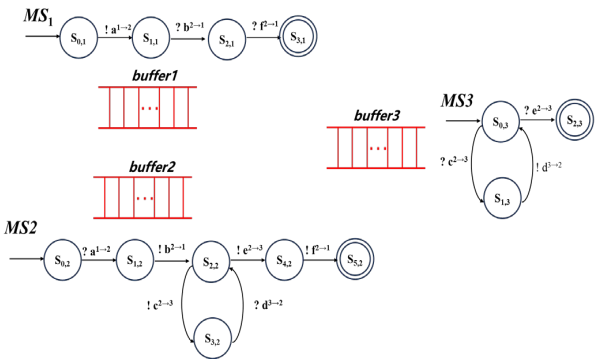


Figure 2: A example of interactions in microservice systems

A. Modeling Method

We chose to use a Labeling Transformation System (LTS) for modeling microservices because of its simplicity and intuitive nature, which has been widely used for modeling in several domains [16].

A labeled transition system is a quintuple $LTS = (S, s_0, F, M, T)$;

1) S is the set of state of the microservice;
2) s_0 is the initial state;
3) F is a set of termination states;
4) M is the set of messages;

$T \subseteq S \times M \times S$ is the transition relation.

The transition relation represents actions and is further defined as three types: send message transition (representing the action of sending a message), receive message transition (representing the action of receiving a) and invisible actions (indicating internal actions).

As a result of the above analysis, we first model individual microservice behaviors using finite LTSs. Next, we model the asynchronous composition among microservices using k-bounded buffers.

Definition 1. (Message Set)[17]: A message set is a tuple(M, MS_s, TX, RX), where:

1) M is a finite collection of messages;

2) MS_s denotes a microservice system, where $MS \geq 1$, denotes the number of participating microservices, MS_s=$\{MS1, MS2, \ldots, MSn\}$;

3) TX and RX denote the send action and receive action, respectively, for associating messages $m \in M$ to MS_s.

Definition 2.(Microservice)[17]: A microservice is represented as an LTS $MS \equiv (S, s, F, \Sigma, T)$,where:

1) S is a finite set of states.

2)s_0 denotes initial state of microservice system;

3) $F \subseteq S$ is a finite set of termination states;

4) $\Sigma \equiv \Sigma^{!} \cup \Sigma^{?}$ is a collection of sending and receiving message sets.

5) $T \subseteq S \times (\Sigma \cup \{\tau\}) \times S$ is the transition relation.

In each microservice, we associate the message $m \in \Sigma$ with communication semantics. Specifically, the send message $m \in \Sigma^{!}$ as $!m$ and the receive message $m \in \Sigma^{?}$ as $?m$. Microservices interact through binary communication on the same messages. Thus, we assume that each message is associated with two different microservices: $\forall i \Rightarrow \Sigma_i^{!} \cap \Sigma_i^{?} = \varnothing$.

Representing final states in the microservice model is crucial. Firstly because microservices may not necessarily have a final state due to the communication cycles, and secondly it is significant when we use a microservice's final state to reason about the final state of microservice systems.

In a microservice , a transition $t \subseteq T$ have three types:

1) A send transition $(s_i, !m^{i \to j}, s_j)$, denotes denotes a microservice sends out a message, where $m^{i \to j} \in \Sigma$;

2) A receive transition $(s_i, ?m^{i \to j}, s_j)$, denotes a microservice consumes a message ,where $m^{i \to j} \in \Sigma$;

3) A ε transition denotes the internal action of the microservice MS, which is not visible by default.

Next, we define the k-bounded asynchronous composition through asynchronous communication and bounded FIFO buffers. Asynchronous communication is non-blocking compared to synchronous communication[18]. This means the send and receive messages do not have to take place in a single step. When the sender sends out the message, it does not require immediate consumption.

There are two types of communication in microservices systems: peer-to-peer communication and mailbox communication. The difference between the two is the number of buffers, which is greater for peer-to-peer communication than for mailbox communication. Therefore, this paper focuses on mailbox communication based on FIFO buffers. As shown in Figure 1, microservice MS_1 sets up *buffer1* to store messages from microservice MS_2 and microservice MS_3 .

The microservices system can be composed of microservices through an asynchronous combination with FIFO-style buffers. Each microservice is configured with a FIFO buffer B_i. Each microservice MS_i can read the message m stored at the head of its FIFO buffer B_i, send the message to the end of buffer B_i of another microservice (If the buffer is not filled with messages), or perform internal operations.

Definition 3(FIFO Buffer): The FIFO-style buffer is a message queue where new messages are added to the end of the queue, and the next message to be consumed is retrieved from the front of the queue.

FIFO buffer based mailbox communication is an asynchronous communication. This communication method requires each microservice to have only one buffer to store messages from other microservices. As shown in Figure 1, microservice *MS1* sets up *buffer1* for storing messages sent to microservice *MS1* by microservice *MS2* and microservice *MS3*.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

In the following, we will define the *k*-bounded microservice system $MS_a^k = (S_a^k, s_a^0, F_a^k, \Sigma_a^k, T_a^k)$ with each FIFO buffer of size *k*. If the receiving microservice's FIFO buffer has k consumed messages, the send action is blocked.

Definition 4 (k-bounded asynchronous composition): Given a set of microservices $\{MS_1, MS_2, \ldots, MS_n\}$ with $MS_i = (S_i, s_0, F_i, \Sigma_i, T_i)$ and B_i^k is its buffer, $(MS_1, B_1^k)||(MS_2, B_2^k)|| \ldots (MS_n, B_n^k)$ is the labeled transition system where:

1) $S_a^k \subseteq S_1 \times B_1^k \times S_2 \times B_2^k \times \ldots \times S_n \times B_n^k$

2) $s_a^0 \equiv \left(s_1^0, \varepsilon, s_2^0, \ldots, s_n^0, \varepsilon\right)$ (ε where denotes an empty buffer)

3) $F^a \subseteq S_a^k$

4) $\Sigma_a^k \equiv \Sigma_1 \cup \Sigma_2 \cup \ldots \Sigma_n$

5) $T_a^k \equiv S_a^k \times \left(\Sigma_a^k \cup \{\tau\} \times S_a^k\right)$

For $s = (s_1, B_1^k, s_2, B_2^k, \ldots, s_n, B_n^k) \in S_a^k$ and $s' = (s_1', B_1^{k'}, s_2', B_2^{k'}, \ldots, s_n', B_n^{k'}) \in S_a^k$, we have

a） **(send action)** $(s, !m, s') \in T_a^k$ if $\exists i, j \in \{1,2,\ldots,n\}$ where $i \neq j : !m \in \Sigma_i^! \wedge ?m \in \Sigma_j^?$ and $\left|B_j^k\right| < k$,

(i). $(s_i, !m, s_i') \in T_i$,
(ii). $B_j^{k'} = B_j^k \cup \{m\}$,
(iii). $\forall l \in \{1,2,\ldots,n\} : l \neq i \Rightarrow s_l' = s_l$,
(iv). $\forall l \in \{1,2,\ldots,n\} : l \neq j \Rightarrow B_l^{k'} = B_l^k$.

b） **(receive action)** $(s, ?m, s') \in T_a^k$ if $\exists i \in \{1,2,\ldots,n\} : ?m \in \Sigma_i^?$,

(i). $(s_i, ?m, s_i') \in T_i$,
(ii). $m \in B_i^k$,
(iii). $B_i^{k'} = B_i^k - \{m\}$,
(iv). $\forall l \in \{1,2,\ldots,n\} : l \neq i \Rightarrow s_l' = s_l$,
(v). $\forall l \in \{1,2,\ldots,n\} : l \neq i \Rightarrow B_l^{k'} = B_l^k$.

c） **(internal action)** $(s, \tau, s') \in T_a^k$ if $\exists i \in \{1,2,\ldots,n\}$,

(i). $(s_i, \tau, s_i') \in T_i$,
(ii). $\forall l \in \{1,2,\ldots,n\} : l \neq i \Rightarrow s_l' = s_l$,
(iii). $\forall l \in \{1,2,\ldots,n\} : B_l^{k'} = B_l^k$.

The above definition states that the k-bounded asynchronous composition has three types of actions. Specifically, the send action is global behavior, which involves the send message and the buffer of the receiver peer. The receive action is local behavior, which only considers the buffer of the receiver peer. The internal action is unobservable behavior, which only involves the corresponding peer. In this paper, we consider the behaviors of the k-bounded asynchronous composition as the ordering of send actions and internal actions, and ignore the ordering of receive actions. This is because receive actions are considered private behaviors[30].

B. Deadlock-free Property Definition

The deadlock-free property refers to a situation in a concurrent system where there is no occurrence of deadlock among multiple processes or threads. A deadlock is a situation where multiple processes or threads cannot proceed because they are each holding resources the others need. In the presence of a deadlock, processes or threads in the system may end up in an infinite wait, leading to a stalemate where the entire system is unable to continue functioning. The deadlock-free property mean the non-blocking behavior of a microservice system. This property is used to check whether the system has any pending states that cannot perform further operations. The deadlock property mean the non-blocking behavior of microservices systems. This property is used to check if there is a pending state in which no further action can be executed [19]. A microservice system is considered deadlock-free if its initial global state can either reach a global state where each microservice can achieve its termination state, or if it can reach a global state that is devoid of deadlocks[20]. In a microservices system, the definition of deadlock-free qualities involves ensuring that all microservices in the system can continue to run and process requests without falling into a state of perpetual waiting for resources or conditions to be met.

Defining the deadlock-free property first requires defining the set of reachable states.

Definition 5 (Reachable States): Given a set of microservices $\{MS_1, MS_2, \ldots, MS_n\}$ with $MS_i = (S_i, s_i^0, F_i, \Sigma_i, T_i)$ and B_i being its associated bounded buffer, the set of global states reachable of the microservice systems from the initial global state, denoted by $Reachable(s_1^0, s_2^0, \ldots, s_n^0)$ and $\forall i \in \{1,2,\ldots,n\} : \forall (s_i^0, \delta_i, s_i') \in T_i$:

- if $\delta_i = !m^{i \to j}$, $\exists j \in \{1,2,\ldots,n\}$, $j \neq i$, $\exists (s_i, \delta_i, s_i') \in T_i \wedge \exists (s_j, \delta_j, s_j') \in T_j$ such that $\delta_j = ?m^{i \to j}$, then $Reachable(s_1^0, s_2^0, \ldots, s_n^0) = Reachable(s_1^0, s_2^0, \ldots, s_n^0) \cup \{(s_1^0, \ldots, s_i', \ldots, s_n^0)\}$
- if $\delta_i = ?m^{j \to i}$, $\exists j \in \{1,2,\ldots,n\}$, $j \neq i$, $\exists (s_j, \delta_j, s_j') \in T_j \wedge \exists (s_i, \delta_i, s_i') \in T_i$ such that $\delta_j = !m^{j \to i}$, then $Reachable(s_1^0, s_2^0, \ldots, s_n^0) = Reachable(s_1^0, s_2^0, \ldots, s_n^0) \cup \{(s_1^0, \ldots, s_i', \ldots, s_n^0)\}$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

- if $\delta_i = \tau$, then $Reachable(s_1^0, s_2^0, \ldots, s_n^0) = Reachable(s_1^0, s_2^0, \ldots, s_n^0) \cup \{(s_1^0, \ldots, s_i^{'}, \ldots, s_n^0)\}$

Definition 6 (Deadlock-free): Given a set of microservices $\{MS_1, \mathrm{MS}_2, \ldots, \mathrm{MS}_n\}$ with $MS_i = (S_i, s_i{}^0, F_i, \Sigma_i, T_i)$ and B_i is its bounded buffer. From the initial state starting from the fact that any possible sequence of state transitions does not cause the system to enter a deadlock state, i.e., all states are reachable.For $(s_1, s_2, \ldots, s_n) \in DF(TS_a)$, then $\forall(s_1^{'}, s_2^{'}, \ldots, s_n^{'}) \in Reachable(s_1^0, s_2^0, \ldots, s_n^0)$.

IV. Tool Support and Experiments

A. Tool Support

The method in this article is implemented by using Process Analysis Toolkit (PAT)[6]. PAT is an independent framework designed to support the composition, simulation, and reasoning of distributed systems. PAT tool supports the modeling, composition, and simulation of labeled transition systems. PAT has implemented a variety of model checking techniques to meet different properties, such as deadlock- free, divergence- freedom, reachability, and LTL property checking with fairness assumptions. In this paper, we utilize a typical microservices system case study to model microservices using LTS and convert it into CSP# language. In addition, we utilize the PAT model detection tool to verify that t the system satisfies the no deadlock property.

B. Data Set

To evaluate the effectiveness of our approach, we use 15 real-world cases from a database of examples[21], and existing papers (e.g.,[22][23][24][25][26]). The size of all cases is described in Table 1, where PA denotes the number of microservices and MS denotes the number of messages in the case.

Table 1: Cases used in our experiments.

Case	Size	
	PA	MS
Case01	3	6
Case02	2	3
Case03	2	5
Case04	4	5
Case05	4	7
Case06	4	8
Case07	3	6
Case08	3	3
Case09	2	4
Case10	2	2
Case11	4	4
Case12	3	4
Case13	3	3
Case14	3	2
Case15	4	5

Identify applicable funding agency here. If none, delete this text box.

C. Experiment implenment

Our approach is completely automated with the support of PAT tools. First, we implemented an encoder that takes the microservices system as input and outputs the corresponding CSP# process. We generate asynchronous combinations using the PAT simulator in three steps. Figure 3 presents the implementation of the microservice system via FIFO buffers. The first step, we encode microservice models denoted as LTSs into CSP# process algebra (Lines 15-18 in Figure 3), where CSP# is an extension of the Communicating Sequential Processes (CSP). Specifically, we write $buffer!m$ for the message emission which indicates that a message m is sent to the tail of $buffer_i$ if $buffer_i$ is can continue to receive messages; we write $buffer?m$ for the message receive ,which means that message m is consumed from header of $buffer_i$. The second step, we use the channel data type for FIFO buffers (Lines 3-6 in Figure 3). The third step, we use the interactive method in PAT (i.e., |||) to compose a set of microservices are asynchronously combined into a microservice system via buffers..

Secondly, once we have the CSP# process, we use the PAT simulator, defined in an LTS, to generate the state space for the asynchronous interaction behavior of the microservice system. The PAT simulator allows us to analyze and visualize the interaction behavior of the microservice system across different states in detail. For instance, the asynchronous interaction behavior of the microservice system with buffers of size 2, as depicted in Figure 2, is illustrated in Figure 4 using the PAT simulator. This system has a total of 28 states and 46 transitions. Such a detailed state space analysis helps us gain a deeper understanding of the system's behavioral patterns.

Finally, we use the model-checking technique in PAT tool to detect the *Deadlock-free* property of a microservice systems. Specifically, The *Deadlick-free* property is defined as:

$$\#assert\ System_k()\ deadlockfree \quad (1)$$

where *System_k* () denotes the *k-bounded* microservice Systems and *deadlock-free* is a keyword for the *Deadlock-free* property.

We can see in Figure 5 that the result returned after clicking Validate isvalid, representing that the microservice system is deadlock-free, indicating that the microservice not have an interaction error during the interaction. In contrast, if the return is not valid, it means that the microservice systen is deadlock. Returns not only the validation results, but also the runtime and memory usage of the microservice system during the validation process.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

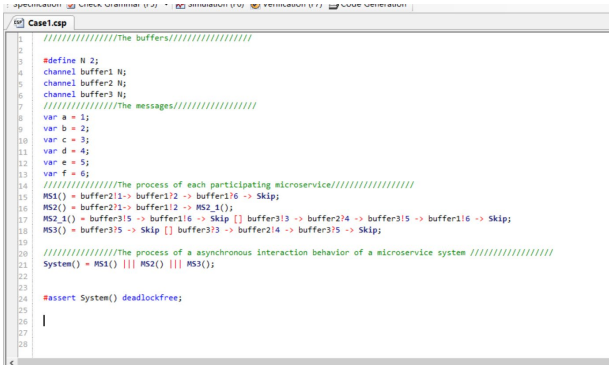


Figure 3: Asynchronous composition via FIFO buffers

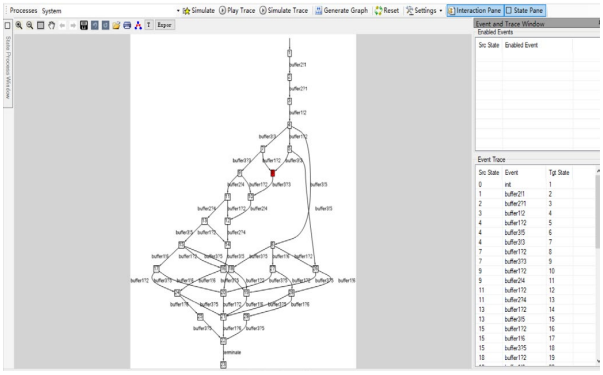
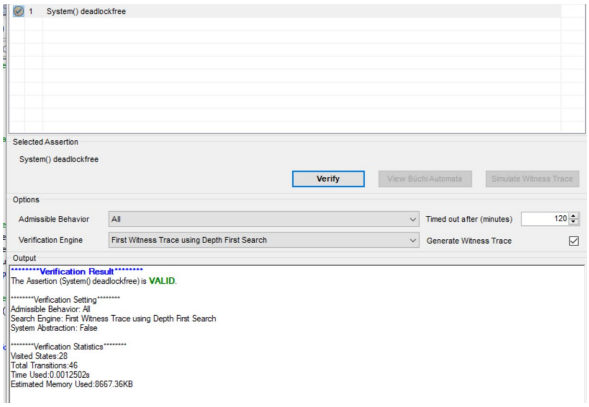
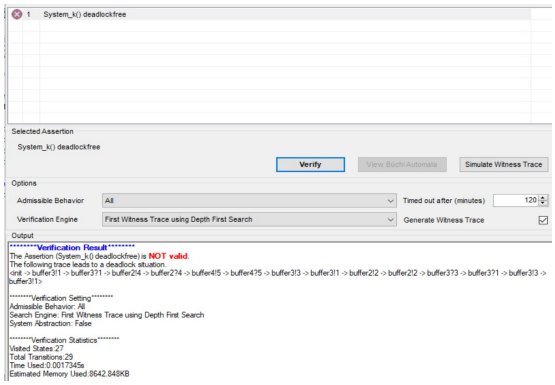


Figure 4: The state space



(a) Deadlock Free



(b) Deadlock

Figure 5: The verification result

D. Experiment Results

Table 2 shows the results of the deadlock analysis. " ✓" denotes that the system is deadlock-freedom and " ✖" denotes that the system is deadlock.

From the table, we can see that Case01, Case02, Case03, Case07, Case09, Case10, Case12, Case15 are deadlock-free systems, indicating that they do not have deadlock problems during interaction, while Case04, Case05, Case06, Case08, Case11, Case13, Case14 do not satisfy the deadlock-free property, and these systems have errors during the interaction process, resulting in deadlocks. By analyzing these systems, we can gain a deeper understanding of the causes of deadlock and provide reference for further optimization of the system. From the table, the higher the number of microservices, the more complex the interaction behaviour of the system is, and the more likely it is to generate interaction errors and deadlocks. For example, Case04, Case05, Case06, Case11. In the experiments of this paper, the buffer size is set to 2. In future work we will compare the interaction behaviour of microservice systems with different buffer sizes. In addition, the average time spent on checking is 3.02s.

Our work is able to detect if the problem of deadlocks occurs during the interaction of the microservice system. All experiments were conducted on the same Windows laptop with a 1.0GHz Intel Core i5 processor and 8GB of Memory.

Table 2: The results of deadlock-freedom analysis.

Case	*Deadlock* checking		
	Time(ms)	memory(kb)	DF
Case01	7.951	8620	✓
Case02	23.02	8652	✓
Case03	0.9828	8659	✓
Case04	1.5026	8659	✖
Case05	1.8504	8676	✖
Case06	2.897	8642	✖
Case07	1.1331	8601	✓
Case08	1.0973	8577	✖
Case09	1.2666	8598	✓
Case10	1.2871	8563	✓

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

Case11	2.3053	8672	×
Case12	2.2544	8713	✓
Case13	1.2661	8569	×
Case14	2.1847	8569	×
Case15	4.5302	11468	✓

V. Conclusion And Future Work

Deadlock-free property is the minimal requirement to ensure the correctness of Microservice systems. This study highlights the importance of deadlock analysis in microservice systems, particularly with bounded buffers. By leveraging formal verification methods like CSP# and tools like PAT, it is possible to model, simulate, and analyze the complex interactions in microservice architectures. This approach not only aids in detecting deadlocks but also contributes to the overall reliability and performance of microservice systems.

In future work, we plan to investigate the deadlock properties of microservice systems using unbounded buffers across various communication models. This analysis will help further understand the implications of different buffering strategies on system reliability and performance. Complex asynchronous interactions between a large number of microservices and infinite increase in buffer lengths will result in a microservice system with an unbounded state space. The unbounded state space makes it undecidable to analyze the interactions of a microservice system based on exhaustive state search. The intricate and varied cyclic structures found within microservice systems give rise to an unending stream of interaction messages. Theoretically, since unbounded buffers have the capacity to hold limitless interaction messages, the range of potential behaviors for a microservice system expands indefinitely. This reality renders traditional modeling techniques inadequate for capturing the full scope of interaction behaviors, prompting the need for innovative modeling and analytical approaches tailored to endless asynchronous interactions. In addition, both synchronous and asynchronous interactions must be considered when modeling and analyzing microservice systems. This is because the behavioral interaction between microservices involves a large number of communication policies, which exhibit different characteristics and challenges in synchronous and asynchronous modes. Therefore, ignoring any of these interaction patterns can lead to inaccurate analysis results. How different microservices communicate and interact with each other not only affects the overall performance of the system, but also adds additional complexity to the modeling and analysis process, requiring a more comprehensive and nuanced approach to address these challenges.

In the future, we will also focus on the interaction behavior of microservices under different communication models, where different communication models lead to different behavioral outcomes. We will focus on the interaction behaviour of microservice systems under different buffers.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 62262063; in part by the key research and development program of Yunnan Province under Grant 202402AD080002-5; in part by the Expert Workstation of Yunnan Province under Grant 202105AF150013; in part by the Yunnan Revitalization Talents Support Plan (XDYC-CYCX-2022-0009).

References

[1] A. Balalaie, A. Heydarnoori and P. Jamshidi, "Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture," in IEEE Software, vol. 33, no. 3, pp. 42-52, May-June 2016.

[2] S. Li, H. Zhang, Z. Jia, Z. Li, C. Zhang, J. Li, Q. Gao, J. Ge, and Z. Shan, "A dataflow-driven approach to identifying microservices from monolithic applications," Journal of Systems and Software, vol. 157, p. 110380, 2019.

[3] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Delta Debugging Microservice Systems with Parallel Optimization," IEEE Transactions on Services Computing, pp. 1-1, 2019.

[4] W. Shih and S. Kaufman, "Netflix in 2011," Harvard Bus. Sch. Case, pp. 615–007, 2014.

[5] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B. C. Ooi, and J. Yang, "Overload Control for Scaling WeChat Microservices," in Proceedings of the ACM Symposium on Cloud Computing, Carlsbad, CA, USA, 2018, pp.

[6] J.Sun,Y.Liu,andJ.S.Dong,"ModelcheckingCSPrevisited:Introducing a process analysis toolkit," in Proc. Int. Symp. Leveraging Appl. Formal Methods Verification Validation, 2008, pp. 307–322.

[7] L. Bao, C. Wu, X. Bu, N. Ren and M. Shen, "Performance Modeling and Workflow Scheduling of Microservice-Based Applications in Clouds," IEEE Transactions on Parallel and Distributed Systems, vol. 30, no. 9, pp. 2114-2129, 2019.

[8] T. Bultan, X. Fu, R. Hull, and J. Su, "Conversation specification: a new approach to design and analysis of e-service composition," in *Proceedings of the 12th international conference on World Wide Web*, 2003, pp. 403-410.

[9] R. Hennicker, M. Bidoit, and T.-S. Dang, "On synchronous and asynchronous compatibility of communicating components," in *International Conference on Coordination Languages and Models*, 2016: Springer, pp. 138-156.

[10] G. Granchelli, M. Cardarelli, P. D. Francesco, I. Malavolta, L. Iovino, G. Salaün, T. Bultan, and N. Roohi, "Realizability of choreographies using process algebra encodings," IEEE Transactions on Services Computing, vol. 5, no. 3, pp. 290-304, 2011.

[11] D. Dahmani, M. Boukala, and H. Montassir, "A petri net approach for reusing and adapting components with atomic and non-atomic synchronisation," *behaviour,* vol. 3, p. 5, 2014.

[12] K. Honda and M. Tokoro, "On asynchronous communication semantics," in Proc. Eur. Conf. Object-Oriented Program., 1991, pp. 21–51.

[13] J. P. Costas, "Synchronous communications," Proc. IRE, vol. 44, no. 12, pp. 1713–1718, Dec. 1956.

[14] F. Dai, H. Chen, Z. Qiang, Z. Liang, B. Huang, and L. Wang, "Automatic analysis of complex interactions in microservice systems," Complexity, vol. 2020, pp. 1–12, 2020.

[15] X. Fu, T. Bultan, and J. Su, "Synchronizability of conversations among web services," *IEEE Transactions on Software Engineering,* vol. 31, no. 12, pp. 1042-1055, 2005.

[16] R. Gorrieri, Process Algebras for Petri Nets- The Alphabetization of Dis tributed Systems, Monographs in Theoretical Computer Science, Berlin, Germany: Springer, 2017.

[17] F.Dai,Q.Mo,Z.Qiang,B.Huang,W.Kou,andH.Yang,"Achoreography analysis approach for microservice composition in cyber-physical-social systems," IEEE Access, vol. 8, pp. 53215–53222, 2020.

[18] M. Ouederni, G. Salaün, and T. Bultan, "Compatibility checking for asynchronously communicating software," in *International Workshop on Formal Aspects of Component Software*, 2013, pp. 310-328: Springer.

[19] F. Dai, G. Liu, X. Xu, Q. Mo, Z. Qiang, and Z. Liang, "Compatibility checking for cyber‐physical systems based on microservices," Software: Practice and Experience, vol. 52, no. 11, pp. 2393-2410, 2022.

[20] L. Bordeaux, G. Salaün, D. Berardi, and M. Mecella, "When are two web services compatible?," in *Technologies for E-Services: 5th International Workshop, TES 2004, Toronto, Canada, August 29-30, 2004, Revised Selected Papers 5*, 2005, pp. 15-28: Springer.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.

[21] L. Akroun and G. Salaün, "Automated verification of automata communicating via FIFO and bag buffers," *Formal Methods in System Design,* vol. 52, pp. 260-276, 2018.

[22] G. Salaün, T. Bultan, and N. Roohi, "Realizability of choreographies using process algebra encodings," *IEEE Transactions on Services Computing,* vol. 5, no. 3, pp. 290-304, 2011.

[23] S. Basu and T. Bultan, "Choreography conformance via synchronizability," in Proceedings of the 20th international conference on World wide web, 2011, pp. 795-804.

[24] M. Güdemann, G. Salaün, and M. Ouederni, "Counterexample guided synthesis of monitors for realizability enforcement," in International Symposium on Automated Technology for Verification and Analysis, 2012: Springer, pp. 238-253.

[25] T. Bultan, "Modeling interactions of web software," in 2nd International Workshop on Automated Specification and Verification of Web Systems (WWV'06), 2006: IEEE, pp. 45-52.

[26] X. Fu, T. Bultan, and J. Su, "Synchronizability of conversations among web services," *IEEE Transactions on Software Engineering,* vol. 31, no. 12, pp. 1042-1055, 2005.

[27]]T. Ueda, T. Nakaike and M. Ohara, "Workload characterization for microservices," In 2016 IEEE International Symposium on Workload Characterization (IISWC), Providence, RI, 2016, pp. 1-10, doi: 10.1109/IISWC.2016.7581269.

[28] S. Wang, Z. Ding and C. Jiang, "Elastic Scheduling for Microservice Applications in Clouds," IEEE Transactions on Parallel and Distributed Systems, vol. 32, no. 1, pp. 98-115, 2021, doi: 10.1109/TPDS.2020.3011979.

[29] Z. Ding and Q. Huang, "COPA: A Combined Autoscaling Method for Kubernetes," *2021 IEEE International Conference on Web Services (ICWS)*, Chicago, IL, USA, 2021, pp. 416-425, doi: 10.1109/ICWS53863.2021.00061.

[30] L. Akroun, G. Salaün, and L. Ye, "Automated analysis of asynchronously communicating systems," in *Model Checking Software: 23rd International Symposium, SPIN 2016, Co-located with ETAPS 2016, Eindhoven, The Netherlands, April 7-8, 2016, Proceedings 23*, 2016, pp. 1-18: Springer.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:03:53 UTC from IEEE Xplore. Restrictions apply.